

Spring Boot Roadmap – Zero to Job-Ready in 8 Weeks

Pre-requisite: Basic Java aani chahiye – OOPs, Collections, Exception Handling, Java 8 features. Nahi aati toh pehle Java roadmap dekho (Java Roadmap already uploaded hai). Feel free to DM @kaash.sde if you have any doubt or confusion.

Week-wise Syllabus (Glossary)

Week 1-2 – Core Spring

1. Dependency Injection (DI)
2. IoC Container
3. @Component, @Service, @Repository, @Controller
4. Beans & Bean Lifecycle
5. Bean Scopes (Singleton, Prototype)
6. Profiles (dev, staging, prod)
7. application.properties / application.yml
8. Auto-configuration
9. Logging (SLF4J + Logback)

Week 3 – REST APIs

1. @RestController & Request Mapping
2. @PathVariable, @RequestParam, @RequestBody
3. DTOs (Data Transfer Objects)
4. Validation (@Valid, @NotNull, @Size, @Email)
5. Exception Handling (@ControllerAdvice)
6. ResponseEntity & HTTP Status Codes
7. Swagger / OpenAPI Documentation
8. Pagination & Sorting

Week 4 – Database (JPA + Hibernate)

1. Entity Mapping (@Entity, @Table, @Id)

2. Relationships (@OneToMany, @ManyToOne, @ManyToMany)
3. JpaRepository (built-in CRUD)
4. Query Methods & @Query (JPQL)
5. Flyway Migrations
6. @Transactional (Propagation, Rollback)
7. N+1 Problem & Fix (JOIN FETCH)
8. Connection Pooling (HikariCP)

Week 5-6 – Security

1. Spring Security Filter Chain
2. JWT Authentication (Access + Refresh Tokens)
3. BCryptPasswordEncoder
4. Role-based Access (@PreAuthorize, @Secured)
5. OAuth 2.0 (Google/GitHub Login)
6. CORS Configuration
7. Rate Limiting

Week 7 – Testing + DevOps

1. JUnit 5 (@Test, Assertions, @ParameterizedTest)
2. Mockito (@Mock, @InjectMocks, when/thenReturn, verify)
3. @WebMvcTest, @SpringBootTest, @DataJpaTest
4. TestContainers (real DB in Docker)
5. Dockerfile (Multi-stage build)
6. docker-compose (App + PostgreSQL + Redis)
7. GitHub Actions CI/CD
8. Deployment (Railway / AWS)

Week 8 – Microservices

1. Service Design & Boundaries
2. Feign Client (sync communication)
3. RabbitMQ / Kafka (async communication)
4. API Gateway (Spring Cloud Gateway)
5. Circuit Breaker (Resilience4j)
6. Service Discovery (Eureka)
7. Distributed Tracing (Micrometer + Zipkin)

Week 1-2 – Core Spring

Concepts to Learn

1. What is Spring Framework vs Spring Boot
 - a. Why Spring Boot exists (auto-configuration, no XML)
 - b. Spring Boot Starter dependencies
2. Project setup with Spring Initializr (start.spring.io)
3. Project structure
 - a. controllers/
 - b. services/
 - c. repositories/
 - d. models/
 - e. config/
4. Dependency Injection (DI)
 - a. IoC Container (ApplicationContext)
 - b. @Component, @Service, @Repository, @Controller
 - c. Constructor injection (ALWAYS use this)
 - d. @Autowired, @Qualifier, @Primary
5. Bean lifecycle
 - a. Creation → Initialization (@PostConstruct) → Destruction (@PreDestroy)
 - b. Bean scopes (singleton, prototype, request, session)
6. Configuration
 - a. application.properties vs application.yml
 - b. @Value annotation
 - c. @ConfigurationProperties (type-safe config)
 - d. Profiles (dev, staging, prod)
7. Annotations deep dive
 - a. @SpringBootApplication (what it combines)
 - b. @Bean, @Configuration
 - c. @Conditional annotations
8. Logging (SLF4J + Logback)
 - a. Log levels (TRACE, DEBUG, INFO, WARN, ERROR)
 - b. Structured logging

Practice

1. Create Spring Boot project from scratch using Initializr

2. Build a service with 3 layers (controller → service → repository) using DI
3. Create custom beans with @Bean in @Configuration
4. Set up different profiles for dev and prod
5. Understand auto-configuration by running with --debug flag

Resources

Resource	Type
spring.io/guides (Getting Started)	Official
Baeldung – Spring Core	Tutorials
Daily Code Buffer – Spring Boot Basics	English YouTube
Telusko – Spring Framework	English YouTube
Concept & Coding – Spring Boot	Hindi YouTube

Week 3 – REST APIs

Concepts to Learn

1. HTTP methods recap (GET, POST, PUT, PATCH, DELETE)
2. @RestController vs @Controller
3. Request mapping
 - a. @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping
 - b. @RequestMapping (class level)
4. Request handling
 - a. @PathVariable (dynamic URL: /users/{id})
 - b. @RequestParam (query params: /users?page=1)
 - c. @RequestBody (JSON body → Java object)
 - d. @ResponseBody
5. DTO pattern (Data Transfer Objects)
 - a. Why not expose entities directly
 - b. Request DTOs vs Response DTOs
 - c. ModelMapper or MapStruct for conversion
6. Validation
 - a. @Valid annotation
 - b. @NotNull, @NotBlank, @Size, @Email, @Min, @Max
 - c. @Pattern (regex validation)
 - d. Custom validators
7. Exception handling

- a. @ExceptionHandler (method level)
- b. @ControllerAdvice (global)
- c. Custom exception classes (ResourceNotFoundException, etc.)
- d. Consistent error response format
- 8. Response handling
 - a. ResponseEntity
 - b. HTTP status codes (200, 201, 204, 400, 401, 403, 404, 500)
- 9. API documentation
 - a. SpringDoc OpenAPI (Swagger UI)
 - b. @Operation, @ApiResponse annotations
- 10. Pagination and sorting
 - a. Pageable interface
 - b. Page response
 - c. @PageableDefault

Practice

1. Build CRUD API for a User entity (all 5 methods)
2. Add input validation with custom error messages
3. Implement global exception handler with consistent error format
4. Add Swagger documentation – test all endpoints from Swagger UI
5. Implement pagination and filtering (GET /users?page=0&size=10&sort=name)

Resources

Resource	Type
spring.io/guides/rest-service	Official
Baeldung – REST with Spring Boot	Tutorials
Amigoscode – Spring Boot REST API	English YouTube
Piyush Garg – Node to Spring comparison	Hindi YouTube

Week 4 – Database (JPA + Hibernate)

Concepts to Learn

1. JPA vs Hibernate

- a. JPA = specification (interface)
- b. Hibernate = implementation
- 2. Entity mapping
 - a. @Entity, @Table
 - b. @Id, @GeneratedValue (strategy: IDENTITY, SEQUENCE, AUTO)
 - c. @Column (name, nullable, unique, length)
- 3. Relationships
 - a. @OneToOne (user ↔ profile)
 - b. @OneToMany / @ManyToOne (user → posts)
 - c. @ManyToMany (students ↔ courses)
 - d. Cascade types (PERSIST, MERGE, REMOVE, ALL)
 - e. Fetch types (EAGER vs LAZY – default to LAZY!)
- 4. Spring Data JPA
 - a. JpaRepository<Entity, ID> (built-in CRUD – no code needed)
 - b. Query methods (findByName, findByAgeGreaterThan, findByStatusAndRole)
 - c. @Query (JPQL – object-oriented SQL)
 - d. @Query (nativeQuery = true – raw SQL)
 - e. Specifications (dynamic queries for complex filters)
- 5. Database migrations
 - a. Flyway (recommended) or Liquibase
 - b. Versioned migration files (V1__create_users.sql)
 - c. Why never use spring.jpa.hibernate.ddl-auto=update in production
- 6. Connection pooling
 - a. HikariCP (default in Spring Boot)
 - b. Configuration (max pool size, timeout)
- 7. Transactions
 - a. @Transactional annotation
 - b. Propagation levels (REQUIRED, REQUIRES_NEW, etc.)
 - c. Isolation levels
 - d. Rollback behavior (rollbackFor)
- 8. N+1 problem
 - a. What it is (1 query for parent + N queries for children)
 - b. How to fix (JOIN FETCH, @EntityGraph, batch fetching)
 - c. How to detect (Hibernate logging: show_sql=true)

Practice

1. Design schema for e-commerce (User, Product, Order, OrderItem – all relationships)
2. Implement all relationship types with proper cascade and fetch

3. Write 10 custom query methods + 5 @Query JPQL queries
4. Set up Flyway migrations (create tables, add columns, add indexes)
5. Find and fix N+1 query problem using JOIN FETCH

Resources

Resource	Type
spring.io/guides/accessing-data-jpa	Official
Baeldung – JPA/Hibernate	Tutorials
Vlad Mihalcea blog	JPA expert (best for Hibernate)
Thorben Janssen – Hibernate Tips	English YouTube

Week 5-6 – Security

Concepts to Learn

1. Spring Security architecture
 - a. Security filter chain (how requests are intercepted)
 - b. Authentication vs Authorization
 - c. SecurityFilterChain bean (new way – no WebSecurityConfigurerAdapter)
2. Password security
 - a. BCryptPasswordEncoder (never store plain text!)
 - b. Password strength rules
3. JWT Authentication
 - a. What is JWT (header.payload.signature)
 - b. Flow: login → generate token → send in header → validate on every request
 - c. Access tokens (short-lived: 15 min)
 - d. Refresh tokens (long-lived: 7 days)
 - e. Token rotation (new refresh token on each use)
4. Implementation steps
 - a. JwtUtil class (generate, validate, extract claims)
 - b. JwtAuthenticationFilter (OncePerRequestFilter)
 - c. AuthController (signup, login, refresh)
 - d. SecurityConfig (configure filter chain, permit/authenticate paths)
5. Role-based access control (RBAC)
 - a. @PreAuthorize("hasRole('ADMIN')")
 - b. @Secured("ROLE_USER")

- c. Method-level security
- 6. OAuth 2.0 (concept + basic implementation)
 - a. Google login flow
 - b. GitHub login flow
 - c. Spring Security OAuth2 Client
- 7. CORS configuration
 - a. @CrossOrigin (controller level)
 - b. Global CORS config (WebMvcConfigurer)
 - c. Allowed origins, methods, headers
- 8. Rate limiting
 - a. Bucket4j or custom implementation
 - b. Per-user vs per-IP limiting
- 9. Security best practices
 - a. Never expose sensitive data in responses
 - b. Input validation on all endpoints
 - c. HTTPS in production
 - d. Security headers (Content-Security-Policy, X-Frame-Options)

Practice

1. Implement complete signup/login flow with JWT
2. Add refresh token rotation
3. Role-based access (ADMIN can delete, USER can only read)
4. Implement Google OAuth login
5. Add CORS configuration for React frontend
6. Add rate limiting on login endpoint (prevent brute force)

Resources

Resource	Type
spring.io/guides/securing-web	Official
Baeldung – Spring Security	Tutorials
Amigoscode – Spring Security + JWT	English YouTube
Daily Code Buffer – Spring Security	English YouTube
Ali Bouali – Full Security Tutorial	English YouTube

Week 7 – Testing + DevOps

Concepts to Learn

1. Testing pyramid

- a. Unit tests (fast, many – test individual methods)
- b. Integration tests (medium – test layers together)
- c. E2E tests (slow, few – test full flow)

2. JUnit 5

- a. `@Test`, `@DisplayName`
- b. `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`
- c. Assertions (`assertEquals`, `assertTrue`, `assertThrows`, `assertNotNull`)
- d. `@ParameterizedTest` (test with multiple inputs)
- e. `@Nested` (group related tests)

3. Mockito

- a. `@Mock` – create mock object
- b. `@InjectMocks` – inject mocks into class under test
- c. `when().thenReturn()` – define mock behavior
- d. `verify()` – check method was called
- e. `ArgumentCaptor` – capture arguments passed to mock
- f. `@MockBean` (Spring Boot – mock in application context)

Integration testing

- a. `@SpringBootTest` (full context)
- b. `@WebMvcTest` (controller layer only)
- c. `@DataJpaTest` (repository layer only)
- d. `MockMvc` (test HTTP endpoints without server)
- e. `TestContainers` (real PostgreSQL in Docker for tests)

5. Test coverage

- a. JaCoCo plugin
- b. Aim for 70-80% coverage
- c. Focus on business logic, not getters/setters

6. Docker

- a. Dockerfile for Spring Boot
 - i. Multi-stage build (build → run)
 - ii. Use `eclipse-temurin:17-jre-alpine` (small image)
- b. `docker-compose.yml`
 - i. App + PostgreSQL + Redis
 - ii. Environment variables
 - iii. Health checks
 - iv. Volume for database persistence

- c. .dockerignore file
- 7. CI/CD with GitHub Actions
 - a. Workflow file (.github/workflows/ci.yml)
 - b. Steps: checkout → setup Java → test → build → deploy
 - c. Run tests on every push/PR
 - d. Deploy on merge to main
 - e. Secrets management (database URL, JWT secret)
- 8. Deployment
 - a. Railway (simplest – connect GitHub, auto-deploy)
 - b. AWS (EC2 + RDS – more control)
 - c. Environment variables in production
 - d. Health check endpoint (/actuator/health)

Practice

1. Unit test service layer – mock all repositories
2. Integration test controllers with MockMvc
3. Set up TestContainers for real DB tests
4. Achieve 75%+ coverage on your e-commerce project
5. Write Dockerfile (multi-stage) + docker-compose
6. Set up GitHub Actions (test on push, deploy on merge)
7. Deploy on Railway with PostgreSQL add-on

Resources

Resource	Type
JUnit 5 docs	Official
Baeldung – Testing in Spring Boot	Tutorials
TechWorld with Nana – Docker	English YouTube
Amigoscode – Docker + Spring Boot	English YouTube
GitHub Actions docs	Official

Week 8 – Microservices Basics

Concepts to Learn

1. Monolith vs Microservices

- a. When monolith is fine (small team, early stage)
- b. When to break into microservices (scale, team independence)
2. Service design
 - a. Single responsibility per service
 - b. Database per service
 - c. Service boundaries (by business domain)
3. Inter-service communication
 - a. Synchronous: REST (RestTemplate, WebClient, Feign Client)
 - b. Asynchronous: Message queues (Kafka, RabbitMQ)
 - c. When to use sync vs async
4. API Gateway
 - a. Spring Cloud Gateway
 - b. Routing, load balancing, rate limiting
 - c. Authentication at gateway level
5. Service Discovery (concept)
 - a. Eureka (Netflix OSS)
 - b. Why services need to find each other dynamically
6. Message Queues
 - a. RabbitMQ basics (producer → exchange → queue → consumer)
 - b. Kafka basics (producer → topic → partition → consumer group)
 - c. Use cases: email sending, notifications, order processing
7. Circuit Breaker
 - a. Resilience4j
 - b. Why: prevent cascading failures
 - c. States: CLOSED → OPEN → HALF-OPEN
8. Distributed tracing (concept)
 - a. Micrometer + Zipkin
 - b. Trace requests across services
9. Configuration management
 - a. Spring Cloud Config (centralized config)
 - b. Config per environment

Practice

1. Break e-commerce into 3 services (User, Product, Order)
2. Implement REST communication between services (Feign Client)
3. Add RabbitMQ for order → email notification flow
4. Set up API Gateway (route /api/users → user-service, /api/products → product-service)
5. Add circuit breaker on external service calls

6. Docker Compose with all services running together

Resources

Resource	Type
spring.io/microservices	Official
Baeldung – Spring Cloud	Tutorials
Java Brains – Microservices	English YouTube

Resource	Type
Daily Code Buffer – Microservices	English YouTube
Concept & Coding – System Design	Hindi YouTube

Projects (Build These)

#	Week	Project	What It Proves
1	Week 2	Todo API (CRUD + DI)	Spring basics, layered architecture
2	Week 4	Blog API (Users + Posts + Comments + Pagination)	JPA relationships, query methods
3	Week 6	E-commerce Backend (Auth + Cart + Orders + Payment concept)	Security, full CRUD, production ready
4	Week 7	Above project – tested + Dockerized + deployed	DevOps skills, CI/CD
5	Week 8	E-commerce split into microservices	Distributed systems thinking

Interview Topics from Spring Boot

Topic	Frequency
Dependency Injection – IoC kaise kaam karta hai	Every interview
@Component vs @Service vs @Repository vs @Controller	Every interview
Spring Boot auto-configuration	Very High
@Transactional – propagation, rollback	Very High

Spring Security – JWT flow, filter chain	Very High
JPA/Hibernate – N+1 problem, LAZY vs EAGER	Very High
Bean scopes – singleton vs prototype	High
Exception handling – @ControllerAdvice	High
Microservices – when to use, communication patterns	High
Testing – @WebMvcTest vs @SpringBootTest	High

8-Week Summary

Week	Focus	Key Skill
1-2	Core Spring	DI, IoC, Beans, Configuration
3	REST APIs	Controllers, Validation, Error Handling, Swagger
4	Database	JPA, Hibernate, Relationships, Migrations
5-6	Security	JWT, OAuth, RBAC, CORS
7	Testing + DevOps	JUnit, Mockito, Docker, CI/CD, Deploy
8	Microservices	Service communication, Gateway, Queues

Tips

1. **DI samjho pehle.** Spring Boot ka poora magic DI pe based hai. Agar IoC container nahi samjhe toh sab annotations magic lagenge.
2. **Annotations ratta mat lagao.** Samjho ki kya trigger hota hai jab @Autowired lagata ho. Debug karo, logs padho.
3. **N+1 problem seekho.** Ye production bugs ka #1 source hai Spring Boot me. show_sql=true karke har query dekho.
4. **Security skip mat karo.** Har company auth puchti hai. JWT flow tumhe neend me bhi aana chahiye.
5. **Test likho.** Companies love candidates who write tests. JUnit + Mockito = non-negotiable.
6. **Deploy karo.** Localhost pe project = invisible. Live deploy karo. Swagger docs link resume pe daalo.
7. **Microservices over-engineer mat karo.** Pehle monolith solid banao. Microservices tab socho jab scale chahiye.

Beyond the Degree – 8 weeks, Spring Boot zero to production. Core Spring → APIs → Database → Security → Testing → Microservices. Yehi path hai backend developer job tak.

@Kaash.sde